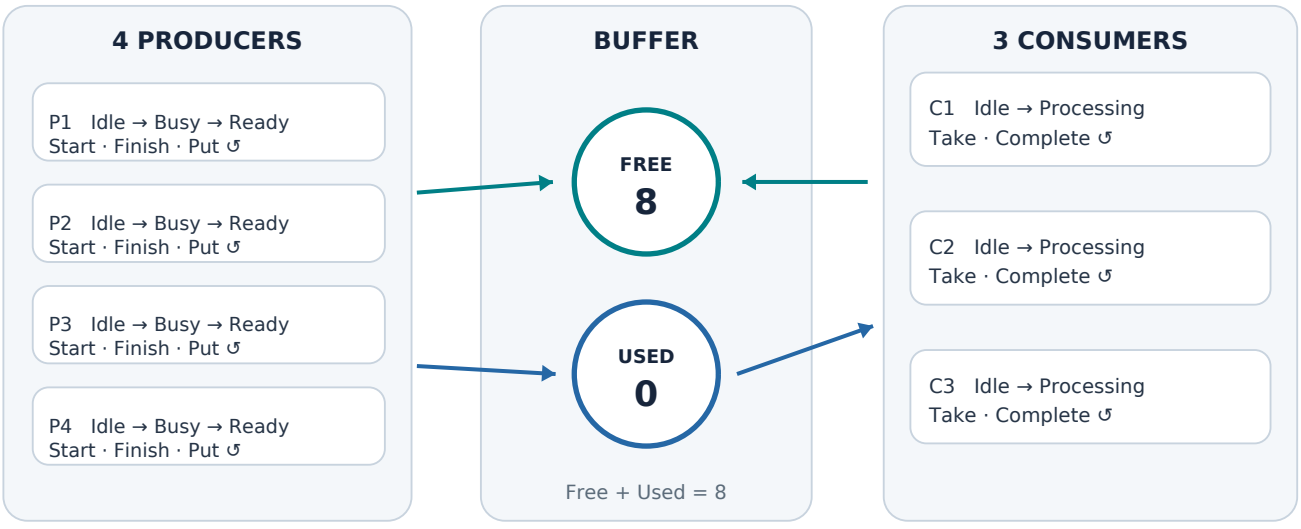
SVG-DOM-GRAPH / DEMO DESIGN

Petri Net: Bounded Producer–Consumer System

A deterministic place/transition demonstration of contention, capacity limits, blocking and re-enabling.

SPECIFICATION	MODEL	STATUS
Revision 3 · 4 November 2026	4 producers · 3 consumers · 8 slots	Phase 0 ready; not started
Target: svg-dom-graph 0.2.47	20 places · 18 transitions · 50 arcs	Demo-only; no new crate API assumed



Aggregate arrows show resource flow, not every one of the 50 canonical arcs.

Figure 1. Conceptual system overview. The canonical place/transition arc list is defined in §3; the overview intentionally condenses actor-local arcs.

Central principle. The bounded buffer is enforced by **two complementary resource places**: Free and Used. The Petri net itself, rather than a UI-side capacity check, prevents a producer from putting an item into a full buffer or a consumer from taking an item from an empty one.

Deliverable: a demo-local Rust engine, a fixed topology rendered using existing Scene nodes and edges, three reproducible scripted traces, manual firing, reversible history and CDP-verifiable accessibility.

1. Purpose and scope

Implement a visually inspectable, **untimed ordinary P/T Petri net** of a shared bounded buffer. Four producer workers prepare items independently and place them into an eight-slot buffer. Three consumer workers remove items and process them independently. Every event is an atomic firing of an enabled transition.

1.1 Required demonstrations

• **Backpressure:** a producer can finish making an item but cannot Put while Free = 0; it stays in Ready.

• **Empty-buffer blocking:** an idle consumer cannot Take while Used = 0.

• **Capacity and exclusivity:** Free + Used = 8 at every reachable marking; no two firings can claim the same last free slot.

• **Resource-driven enabling:** Take frees a slot and enables a previously blocked Put; Put fills a slot and enables previously blocked Takes.

• **True interleaving:** independent workers can be in different states; the UI exposes all currently enabled transitions and why each other one is disabled.

1.2 Deliberate boundaries

This is an **ordinary, uncoloured, untimed P/T net**. The buffer stores counts, not distinct item identities, queue order, FIFO semantics, production times, priorities or stochastic service rates. Playback timing is a presentation rate and does **not** change net semantics. A scheduler chooses among already enabled firings; the engine never invents an extra inhibition rule.

The stage uses a fixed, realistically sized model. Capacity and worker counts are not user-editable in v1. Separate smaller models may be constructed by native tests. No dynamic graph editing and no live concurrency threads are required.

1.3 Repository alignment

The current repository structures demo logic beneath `demo-app/src/`, registers it through `demo_gallery!`, and uses pure Rust logic independent of the browser. Existing demos show the patterns for a fixed outer SVG stage, scenario switching, pure projections, a controller and a deterministic manual clock. Reuse those conventions rather than introducing a framework.

> **Phase 0 gate.** Prototype worker-local ring layouts, shared Put/Take bar-side anchors, seven spokes at each resource hub and app-owned overlay event targets in a browser. Do not commit 50-arc geometry before inspecting the rendered result at 100%.

2. Model at a glance

The token game begins with all four producers Idle, all three consumers Idle, Free = 8 and Used = 0. Every worker owns its own local state places. The buffer resource places are shared among all producers and consumers.

CATEGORY	PLACES	TRANSITIONS	ARCS
4 producers	4 × (Idle, Busy, Ready) = **12**	4 × (Start, Finish, Put) = **12**	4 × (2 + 2 + 4) = **32**
3 consumers	3 × (Idle, Processing) = **6**	3 × (Take, Complete) = **6**	3 × (4 + 2) = **18**

CATEGORY	PLACES	TRANSITIONS	ARCS
Shared buffer	Free, Used = **2**	None	Included in actor arcs
Totals	**20**	**18**	**50**

2.1 Meaning of local places

Producer Busy means the worker is preparing one item. **Ready** means that item has been prepared locally but is not yet in the buffer. A blocked producer is specifically Ready with Free = 0. Producer Idle means it can start another item.

Consumer Processing means a buffer item was removed and the slot was immediately released. A blocked consumer is specifically Idle with Used = 0. Processing is not counted in Used; buffer occupancy measures only items still waiting in the bounded buffer.

2.2 Initial marking

```
for i in 0..4: Producer[i] = (Idle=1, Busy=0, Ready=0)
for j in 0..3: Consumer[j] = (Idle=1, Processing=0)
Buffer = (Free=8, Used=0)
```

Actor control places are 1-safe; the two buffer places are 8-bounded. Every arc has weight 1. There are no read, reset, inhibitor or capacity arcs.

3. Canonical arcs and formal semantics

The following table is normative. Each row is instantiated once per worker ID; a comma means one input/output arc per named place. No unspecified arc is implied.

TRANSITION	INPUT PLACES (PRE)	OUTPUT PLACES (POST)	WHEN IT CAN FIRE
Start[i]	P[i].Idle	P[i].Busy	Producer is idle
Finish[i]	P[i].Busy	P[i].Ready	Item preparation is active
Put[i]	P[i].Ready, Buffer.Free	P[i].Idle, Buffer.Used	Prepared item and a free slot
Take[j]	C[j].Idle, Buffer.Used	C[j].Processing, Buffer.Free	Idle consumer and stored item
Complete[j]	C[j].Processing	C[j].Idle	Consumer finishes its current item

3.1 Enabling and firing

Let M be the non-negative marking vector and Pre/Post the incidence functions. A transition t is enabled exactly when M[p] ≥ Pre[p,t] for **every** place p. Firing an enabled t produces M′[p] = M[p] − Pre[p,t] + Post[p,t]. A single event checks every input, computes a full successor marking, validates it and commits atomically. Disabled firing is a no-op and returns a typed reason.

```
enabled(M,t) = all(M[p] >= Pre[p,t] for p in P)
if !enabled(M,t): return Disabled(missing_inputs)
next[p] = M[p] - Pre[p,t] + Post[p,t]  // all p
validate(next); return Fired(next)
```

3.2 Derived explanations, never independent state

The engine computes **BlockedFull[i]** iff P[i].Ready = 1 and Buffer.Free = 0; **BlockedEmpty[j]** iff C[j].Idle = 1 and Buffer.Used = 0. A Put lacking Ready is simply waiting for a producer, not blocked by capacity. A Take lacking Idle is processing, not blocked by emptiness. When a transition has multiple missing inputs, the inspector lists all missing places and highlights the local actor prerequisite first.

Recompute the entire enabled set E(M) after every event. Newly enabled transitions are E(M′) − E(M); newly disabled are E(M) − E(M′). The event panel states these changes explicitly rather than inferring them from animation alone.

4. Conservation laws and safety

INVARIANT	EQUATION	WHY IT MATTERS
Producer exclusivity	P[i].Idle + P[i].Busy + P[i].Ready = 1	One outstanding item per producer, at most
Consumer exclusivity	C[j].Idle + C[j].Processing = 1	One in-flight item per consumer, at most
Capacity conservation	Buffer.Free + Buffer.Used = 8	The buffer never under- or over-fills
Buffer accounting	PutCount − TakeCount = Buffer.Used	No ghost items in the shared buffer
Processing accounting	TakeCount − CompleteCount = Σ C[j].Processing	No ghost/duplicated consumer work

The last two equations use event counters stored with the trace, initially zero. They are audit invariants, not extra net places; a page displaying them must derive the values from the same event history. All place counts are integers ≥ 0.

4.1 Why capacity is structural

A Put requires and consumes one Free token and creates one Used token. A Take does the reverse. When Used = 8, Free is necessarily zero, so every Put is disabled even if any number of producers are Ready. With Free = 1 and two Ready producers, both Put transitions are initially enabled, but firing one consumes the sole Free token and disables the other. One-transition-at-a-time firing is mandatory in v1.

4.2 Concurrency claims

Enabling describes possible next events, not an obligation to fire. The absence of a global deadlock does not imply fairness or freedom from starvation under an arbitrary scheduler. The fixed scripted scenarios demonstrate observable blocking and re-enabling, not a guarantee about real-time throughput.

> **Verified state-space target.** An independent breadth-first traversal of the fixed net reaches exactly **5,832 markings**, equal to 3^4 × 2^3 × 9. A reverse traversal of the reachable graph from the initial marking also reaches all 5,832, so the initial marking is a **home marking**. The net has no dead marking. Pin all three exact results; event counters are excluded.

5. Required scenario fixtures

Three deterministic, immutable scripted traces form the baseline. Each event is generated by firing the canonical engine; scripts never mutate a marking. Snapshot 0 is the initial marking and event k produces snapshot k. Scripted traces are immutable even when the viewer enters manual mode.

SCENARIO	FIRINGS	SNAPSHOTS	PURPOSE
Walkthrough	50	51	Full/empty blocking and resource release
Contention	28	29	Two producers claim the final available slot
Interleaving	30	31	Busy, Ready and Processing overlap across workers

5.1 Walkthrough: fill, block, release, drain

Eight complete Start/Finish/Put triples fill the buffer (producers 0,1,2,3 twice); the remaining events exercise blocked producers, their recovery, a full drain, blocked consumers and newly enabled Takes.

INDEX	LAST EVENT	USED / FREE	OBSERVATION
0	Initial	0 / 8	All 3 Takes disabled; 4 Starts enabled
3	Put[0]	1 / 7	All 3 Takes become enabled
24	Put[3]	8 / 0	Full: no Ready producer yet
26	Finish[0]	8 / 0	Blocked producer 0: Put[0] lacks Free
27	Take[0]	7 / 1	Released: Put[0] becomes enabled
28	Put[0]	8 / 0	Last available slot consumed again
45	Complete[1]	0 / 8	Empty: all idle consumers cannot Take
47	Finish[1]	0 / 8	Put[1] enabled; Takes remain blocked
48	Put[1]	1 / 7	All 3 Takes enabled again
49	Take[1]	0 / 8	C1 Processing; C0/C2 wait on empty
50	Complete[1]	0 / 8	All consumers Idle again

5.2 Contention: two producers, last free slot

Seven complete producer triples are followed by Ready producers 0 and 1 competing for the eighth slot. A subsequent Take makes the losing Put newly enabled.

INDEX	LAST EVENT	USED / FREE	OBSERVATION
21	Put[2]	7 / 1	No Ready producer yet
25	Finish[1]	7 / 1	Both Put[0] and Put[1] enabled
26	Put[0]	8 / 0	Put[1] newly disabled; P1 blocked
27	Take[0]	7 / 1	Put[1] newly enabled; P1 unblocked
28	Put[1]	8 / 0	P1 consumes released slot

5.3 Interleaving: overlapping worker states

This third script is **not** a set of complete producer triples or consumer pairs. Producers can be Busy together, one can be Ready while others are Busy, and several consumers can be Processing simultaneously. It returns to the exact initial marking after 30 firings. State strings are in ID order: producers I=Idle, B=Busy, R=Ready; consumers I=Idle, P=Processing.

INDEX	LAST EVENT	USED / FREE	PRODUCERS	CONSUMERS
0	Initial	0 / 8	IIII	III
2	Start[1]	0 / 8	BBII	III
5	Finish[1]	0 / 8	RRBI	III
8	Start[3]	0 / 8	IRBB	PII
11	Take[1]	0 / 8	IIRB	PPI
12	Finish[3]	0 / 8	IIRR	PPI
14	Take[2]	0 / 8	IIIR	PPP
22	Take[1]	1 / 7	RIII	PPP
30	Complete[2]	0 / 8	IIII	III

At index 8, one producer is Ready, two are Busy and one consumer is Processing. At index 14 all three consumers are Processing while producer 3 remains Ready; at index 22 all three process while producer 0 is Ready. **Used never exceeds 1** in Interleaving; it tests simultaneous local work, not buffer saturation. Walkthrough and Contention remain the normative full/empty/capacity fixtures. All values and enabled-set deltas are asserted from the ordinary engine.

5.4 Normative named checkpoints

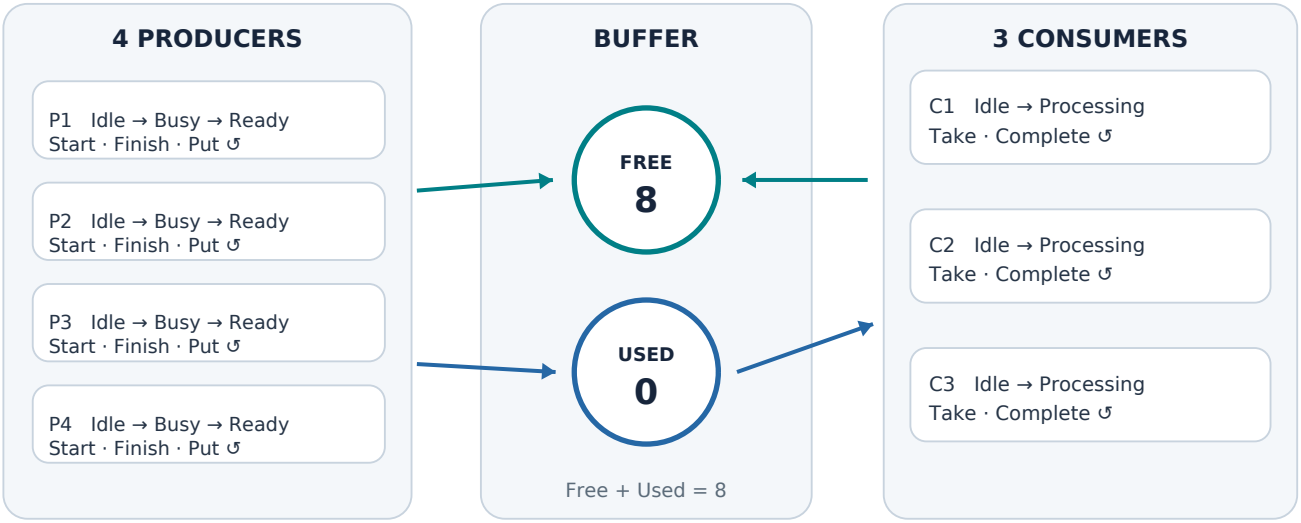
Checkpoint names are exact **snapshot indices**, never event indices. Selecting one performs an absolute seek, not a replay. These are the only named checkpoints in Revision 3:

SCENARIO	CONTROL LABEL → SNAPSHOT INDEX
Walkthrough	Full → **24**; Blocked → **26**; Released → **27**; Empty → **45**
Contention	Both ready → **25**; Last slot taken → **26**; Re-enabled → **27**
Interleaving	Two busy → **2**; Mixed work → **8**; Two processing → **11**; Three processing → **14**; Home again → **30**

Named checkpoints are **unavailable in Manual mode**; manual snapshots are a different local coordinate space and must not be interpreted as scripted indices. The checkpoint selector stays visible but is aria-disabled and explains “Return to script to use checkpoints.”

6. Visual design and explanation

Use real Scene nodes and directed add_edge_with connectors for all 50 unit arcs. Places are circles, transitions are visually narrow rectangles or (if the measured public API cannot produce that aspect ratio) compact labelled rectangular gates. The four producer rings are to the left, Free above Used in the centre, and the three consumer rings to the right. Keep this **same left / centre / right ordering** in both Figures 1 and 2 and in the implemented view.



Aggregate arrows show resource flow, not every one of the 50 canonical arcs.

Figure 2. The same conceptual arrangement as Figure 1: producer-local cycles on the left, Free over Used in the centre, consumer-local cycles on the right. The aggregate arrows are illustrative, not substitutes for the 50 arcs.

6.1 Readable markings and states



Figure 3. Persistent buffer-state display. Occupancy is shown with text as well as a filled bar.

- **Places:** token 0 is an empty actor place; token 1 is a solid dot; Free and Used show integer counts 0–8 without overlapping dots.
- **Transitions:** enabled is a solid outlined gate; disabled is dashed/faded; newly enabled is a temporary double outline; fired receives a strong focus ring and status text.
- **Blocking:** a Ready producer at Free=0 says “Waiting: buffer full”; an Idle consumer at Used=0 says “Waiting: buffer empty”. The inspector reports each missing input.
- **Edges:** arrowheads show direction, every arc has weight 1, and focus reveals all four incident arcs of a selected Put or Take. Visibility and state never rely on colour alone.

6.2 Worker-local ring layout

Use a **ring/loop** in each producer lane rather than laying Idle → Start → Busy → Finish → Ready → Put in a single horizontal row. Put → Idle returns around its own ring without a long cross-lane edge. Each consumer also forms a ring: Idle → Take → Processing → Complete → Idle. Local arcs are routed inside the worker zone and buffer arcs in separately allocated central corridors. Worker rings and labels have fixed coordinates in all 111 scripted snapshots.

6.3 Shared rectangular bar-side anchors

Put and Take each have four incident arcs. Sharing a rectangle side is permitted if endpoints are separately pinned. Use with_from_side / with_to_side, combined with with_from_position / with_to_position on **rectangular** nodes. Position fractions 0.25 and 0.75 distinguish two arcs sharing one side. The following is a **Phase 0 candidate** to be measured, not a claim that all corridors are already clear:

GATE	INPUT 1	INPUT 2	OUTPUT 1	OUTPUT 2
Put[i]	Ready → North	Free → East @ 0.25	East @ 0.75 → Used	South → Idle
Take[j]	Idle → North	Used → West @ 0.75	West @ 0.25 → Free	South → Processing

The upper resource hub is Free and the lower is Used. For **both** Put and Take, the Free-connected endpoint is the **0.25** (upper) anchor and the Used-connected endpoint is the **0.75** (lower) anchor of the shared side. In particular, Take receives Used at West@0.75 and sends Free from West@0.25: reversing those two creates an avoidable crossing at each Take. Verify that 0.25 maps to the upper physical endpoint in the current library, that the two anchor coordinates differ, and that arrowheads remain legible. Rectangular gates must **never** receive ellipse clock positions (ClockOnRectangle).

6.4 Buffer-hub spokes and crossing risk

The seven arcs incident to each central circle use distinct directions. A testable initial fan-out is four producer-facing clock positions 40/44/48/52 and three consumer-facing positions 9/15/21 (0–59, clockwise from north). These are **straight** circle anchors; an elbow is allowed only at cardinal minutes 0/15/30/45. Avoid fractional side positions on an ellipse. Free → Put and Put → Used may still cross despite separate anchors; use separate labelled corridors and measured geometry rather than promising an impossible crossing-free arrangement. In particular, the highest and lowest producer-facing spokes must not **intersect or graze the opposite resource hub**; Phase 0 measures each hub-to-Put route against the other hub's rendered stroke envelope, including its arrowhead, and records its smallest clearance.

The Phase 0 prototype must inspect both hubs, each four-arc gate, arrow direction, endpoint gaps and the Put → Idle loop at 100% and after zoom/pan. Record signed closest clearances between both resource circles (including visible strokes) and **all** producer-facing spokes; intersections/tangencies fail the gate and require corrected hub positions or clock assignments. At 100%, keep a visible margin for strokes and arrowheads; document the final minimum clearance and test tolerance. If crossings remain distracting, provide Overview and Focused wiring presentations which **retain all 50 real arcs**; Focused emphasizes all inputs/outputs of the selected transition and subdues others. Never make non-P/T or aggregate edges stand in for a canonical arc in the formal graph.

7. Controls and user journeys

A fixed outer SVG stage holds controls above the scene, unaffected by pan or zoom. The graph uses a dedicated demo window, following the existing large-demo convention. A status/inspector panel outside the transformed graph reports the marking, enabled set, input shortages and most recent firing.

CONTROL	NORMATIVE BEHAVIOUR
Scenario selector	Walkthrough (default), Contention, Interleaving. Explicit selection switches only the active trace/projection to snapshot 0, preserving Scene/view/focus. In Manual it discards that branch and announces this in the status line.
Enter Manual	Stops playback and forks from the currently displayed scripted snapshot k. Manual history begins at its own index 0, with inherited marking and cumulative event counts.
Return to script	Exits Manual and restores exactly the originating scenario at fork index k, not the current manual index. The manual branch is discarded.
Fire selected	Present in Manual only (or shown aria-disabled in scripts). Fires the selected enabled transition once; a disabled/absent choice changes no snapshot.
Previous / Next / Restart	Scripted: navigate the immutable history; Restart → index 0. Manual: navigate existing local snapshots; Restart → local index 0 without clearing history.
Play / Pause / speed	Scripted only: 1, 2, 5, 10 events/s, default 2. No looping. At the last snapshot Play is aria-disabled and schedules nothing. In Manual Play is always aria-disabled.
Seek range	Script: indices 0..N; Manual: indices 0..branch tip. Drag updates coalesce to a single render per frame, tagged with history generation.
Named checkpoints	Script only, exact §5.4 mapping. In Manual this control is aria-disabled and explains why; no “manual checkpoint seeking”.
Inspect / focus	Single click, tap or focus alone only selects/inspects; Enter/Space subsequently pressed on a focused transition does fire it in Manual, by design.
Quick manual fire	Mouse double-click, or Enter/Space with an enabled focused transition, fires exactly once in Manual. Touch: tap to select then activate Fire selected; no double-tap requirement.
Overview / Focused	Only visual arc emphasis; no change in state or canonical edge count.

7.1 Manual state-machine and history

Entering Manual saves Origin { scenario_id, fork_index } and a **copy** of the source Snapshot including all counter values. The manual branch has snapshots[0] = that copy, events empty, cursor = 0, tip = 0, and a new history generation. For a successful fire at cursor k, prepare a candidate history which discards events/snapshots after k, appends the successor, and sets cursor to k+1; only publish that candidate after the presentation commits successfully. A rejected/disabled fire changes neither cursor, event list, totals nor tip.

Previous/Next/Restart/seek only browse the existing manual branch, never recompute its events. Return to script restores the immutable origin scenario and **its original fork index**, even if the manual branch has advanced or been rewound. Explicit scenario selection instead activates the chosen scripted trace at index 0, including if it names the origin scenario; if Manual was active, set the visible status to **“Manual branch discarded. [Scenario] loaded at snapshot 0.”** and announce it once in the polite live region. The notice remains until the next successful user action or error replaces it. A fresh entry to Manual starts a fresh branch, not a continuation of the discarded one.

All scenario/mode switches pause playback and invalidate late ticks and seek requests by advancing **playback_epoch** and **history_generation**; a manual firing that truncates a recorded future also advances the history generation. These counters identify history requests, not Scene elements. The scenario selector, history label, range maximum, step indicator and checkpoints are derived from the active history and committed with its projected marking on the **retained Scene**. Manual event count is local; cumulative Put/Take/Complete totals are inherited from its forked source snapshot.

7.2 Selection, firing and playback boundaries

Selection alone is inert for all input devices: clicking, tapping, tabbing or using arrow keys to focus a transition does not fire it. The two-step mouse path is click-to-inspect then Fire selected; mouse double-click fires **once** in Manual, and neither constituent click fires. **Keyboard activation is intentionally different from selection:** after a bar receives focus, pressing Enter or Space fires it once in Manual; movement/focus alone does not. Prevent the Space default scroll and repeated/key-up double activation. **Touch has no double-click gesture requirement:** tap a bar to select, then tap Fire selected. All routes call exactly the same enabling/commit validation and **each successful manual fire** announces its transition and outcome via the polite live region. In scripted mode these gestures cannot fire. Disabled/non-transition attempts do not create a history step. The fired transition stays selected with updated status.

At the end of a script, Play has no effect, stays focusable with aria-disabled, and announces “Already at final snapshot.” It neither loops nor restarts. In Manual, Play is always unavailable (history is user-directed, not a predetermined schedule). Manual Next can browse previously recorded future events but cannot autonomously create new firings. All boundary actions first attempt dirty-page repair.

7.3 Inspector example

The inspector gives: the last transition, tokens consumed/produced, changes in enabled transitions, blocking reasons and buffer accounting. Example: “Take[0] consumed Used and C0.Idle; produced Free and C0.Processing; Put[1] became enabled.” Each successful manual firing issues **one** polite live-region announcement, regardless of whether Fire selected, keyboard activation or mouse double-click caused it. Script playback avoids announcing every event and summarizes milestones and pause/end.

8. Engine, trace and presentation architecture

All Petri-net decisions are implemented in pure Rust, independently of `svg-dom-graph`, `web-sys` and timing. The graphics layer is a projection of a validated marking; a Petri concept must not be inferred from CSS classes.

MODULE	RESPONSIBILITY
net/	Typed PlaceId/TransitionId, static Pre/Post arc table, Marking, enabled(), fire(), invariant validation and error enums.
fixtures.rs	Three deterministic event sequences, fixed checkpoint indices and exact milestone assertions.
trace/	Immutable scripted trace, manual origin/fork metadata, local cursor/tip, counter snapshots, truncation and length budget.
projection/	Pure derived enabled sets, blocked reasons, deltas, narration and presentation attributes.
layout/	Worker-ring coordinates, shared bar-side anchors, seven-spoke resource hubs and measured collision constraints.
scene.rs + page.rs	Build exactly one Scene per demo instance; retain the 20 places, 18 gates and 50 arcs across all scenarios/modes. Project only overlays, controls and status.
controller/	Explicit EnterManual/ReturnToScript/SelectScenario/FireSelected actions; seek, mode-specific playback, lifecycle and dirty repair.
scheduler.rs	Clock abstraction and test-controlled playback, event epochs and cancellation.
mod.rs + chrome.rs	Demo entrypoint, gallery source files, panel events, app-owned SVG/HTML controls and teardown.

8.1 Data contracts

```
struct Net { places: Vec<PlaceSpec>, transitions: Vec<TransitionSpec> }
struct Marking { tokens: Vec<u8> }          // fixed length: 20
struct Event { transition: TransitionId }
struct Snapshot { marking: Marking, totals: Totals, last: Option<Event> }
struct View { enabled: BitSet, newly_enabled: BitSet, blocked: Vec<Reason> }
```

Prefer arrays/enums for the fixed demo and avoid string IDs as simulation keys; view keys can be stable strings for DOM testing. Counters use checked integers (for example u64) and event budgets prevent overflow in practice. A snapshot is authoritative; visual highlights and explanatory text are derived from it.

8.2 Boundaries and limits

Provide typed errors for malformed graph specs (wrong part count, duplicate IDs, arc weight zero, unknown place, non-bipartite arc), disabled attempts and event-budget exhaustion. All three fixed fixture generators are infallible after construction tests; app failures return `Result` and do not panic. A manual branch is capped at 10,000 successful firings and announces when the cap is reached; cap enforcement occurs before state mutation.

9. Rendering and Scene integration

9.1 Use existing APIs

Use `Scene::add_circle_node` / `add_ellipse_node` for places, `add_node_with` for rectangular transition gates, `add_edge_with` with side and fraction options for gate arcs, `node_rect` for precise overlay placement, and the existing view/zoom toolbar. `set_focus` may mark the most recently fired transition. Avoid `set_data_values` on plain circle or bar nodes; the existing API does not provide a live label setter for them.

Create one app-owned overlay group within the transformed Scene content for token glyphs, enabled outlines, blocked tags and test/keyboard targets. Fixed node labels and geometry do not change across firings. Overlay groups are pointer-transparent except for their deliberate hit targets. Before implementation, verify where the content group can safely receive app-owned children and how its transform is applied.

9.2 Consistent update path

Define `project(snapshot, preference)` as a pure presentation. The page exposes `present_full` and `present_diff`, which must produce equivalent element attributes, text and accessibility names for any snapshot, including arbitrary reverse seeks. Do not add or remove core Scene nodes or edges on each event; only mark overlays and update the status.

Create the canonical topology **once** per panel instance. Core element IDs have no history generation: `pn-{instance}-p-{id}`, `pn-{instance}-t-{id}` and `pn-{instance}-a-{id}`. Keep `data-pn-place` and `data-pn-transition` stable and update `data-pn-scenario` on the persistent root when the active script changes. The **history_generation** belongs to asynchronous seeks, playback and branches only; it must never appear in stable SVG IDs.

9.3 Transactional trace and projection switch

Scenario changes **must not** rebuild, detach or replace the Scene. The three immutable traces share the same 20-place/18-transition/50-arc topology and geometry. Pause, cancel pending asynchronous actions and prepare the target snapshot 0 and its projected controls/status. Render the full projection onto the existing Scene and update the scenario indicator and control bounds. Only after every write succeeds publish active_scenario, cursor and history_generation. Preserve the existing `Scene::view()` pan/zoom exactly and retain the logical selected/focused PlaceId or TransitionId (all are valid in every fixture). There is no post-switch node/edge creation or replacement. A failed write preserves the last committed scenario and cursor, stops playback and marks the partly updated page dirty; the next action first fully redraws the committed state. A failed control write follows the same rule. Implementing a scene swap is out of scope unless Phase 0 demonstrates a concrete API limitation and the specification is amended before Phase 1.

10. Failure handling, time and teardown

DOM write failure: a proposed change is committed only after its projected snapshot and status have rendered successfully. A partial write stops playback, leaves the trace/index on the last committed snapshot, records the page as dirty and reports a visible error. The very next action (including a disabled boundary press or selecting the same index) attempts a **full redraw of that committed snapshot first**. If repair fails, no new action proceeds. No speculative counter or marking increment is retained.

Playback: available only before the last snapshot of an immutable scripted history. It never loops; it is disabled throughout Manual mode. Schedule the next tick only after the current snapshot is committed. Each scheduled callback carries **(playback_epoch, history_generation)**. Increment the epoch on every stop: Pause, Prev/Next, manual fire, seek, speed change, scenario switch, error and teardown. A late callback whose token no longer matches is ignored, even though the underlying Scene persists. No backlog is accumulated while the browser is suspended; at most one transition is displayed per accepted tick.

Seek: store the latest requested index; coalesce high-rate range changes into one animation-frame draw. Pending seeks carry history_generation; discard them when scenario/mode changes or a manual branch truncates its future. A failed requestAnimationFrame falls back to a synchronous commit and warns if needed. On resume, navigation preserves no stale animation callback.

Teardown: remove listeners, cancel timers/frames, clear queued actions, invalidate generations, drop app-held Scene/DOM handles and remove any test-only global hooks without removing another instance's hooks. Use weak callback captures when retaining controllers would form an Rc cycle. Do not leave a detached demo running.

10.1 Accessibility contract

• Use roving tabindex or a stable logical focus order. Click/tap and keyboard focus navigation select without firing; pressing Enter/Space on a focused transition **intentionally fires in Manual**. Fire selected is the touch and explicit keyboard/mouse alternative; mouse double-click is optional convenience, not the only path.

• Each place announces name and token count; each transition announces its ID, input tokens, status and missing preconditions. Buffer summary announces "Used N of 8, Free M".

• Manual firing announces each event in a polite live region. Playback suppresses per-event announcements and reports pause, blocked milestones and end only.

• When a control cannot advance, leave it focusable with aria-disabled, announce why, and still attempt dirty repair. Follow the existing SVG controls convention.

• Core Scene elements and DOM IDs do not change on scenario switch. Preserve the logical selected PlaceId/TransitionId and its visible keyboard focus ring; update only its state, labels and announcements.

11. Verification and regression matrix

LEVEL	MANDATORY TESTS
Native: topology	Exactly 20 places, 18 transitions, 50 unique unit arcs; bipartite arc endpoints; initial marking as §2; deterministic place/transition ordering.
Native: rules	For each transition type, test enabled and disabled firings, unchanged marking for refusal, exact token deltas, newly enabled/disabled sets and reason priority.
Native: invariants	After every scripted event, verify all five §4 invariants, no underflow/overflow and single-token actor states.
Native: reachability	BFS reachable markings == **5,832** exactly. Reverse BFS from initial marking reaches exactly the same 5,832 (home marking); zero dead markings; all invariants hold.
Native: fixtures	All named §5.4 indices exact; scripted trace sizes 51, 29 and 31; interleaving states at 2/8/11/14/22/30 and all enabled-set deltas exact.
Native: contention	At snapshot 25 both Puts enabled; firing Put[0] makes Put[1] disabled, then Take[0] enables it; a prohibited simultaneous double-Put cannot overfill.
Native: manual history	Local 0, inherited counters, origin index restoration, branch truncation, disabled no-op, unavailable checkpoints/Play and event budget; scenario-select from Manual emits discard notice.
Projection / layout	Full/diff parity across all fixtures/branches; 0.25 Free and 0.75 Used on both Put and Take; no reversed Take anchors; 7 spokes/hub and measured opposite-hub clearance.
Controller / fake DOM	Partial trace-switch write failure retains old history/cursor and marks dirty; next-action repair, failed repair, stale tick/seek across a history change, timer error.
CDP browser	All 50 real arcs and Take West@0.75 Used / West@0.25 Free; measured extreme-spoke clearance, interleaving states; double-click/Enter/Space/touch Fire selected fire once.
CDP lifecycle	Scenario switches keep the same Scene/IDs and exact zoom/pan; return from Manual retains fork index/focus; branch discard notice; no stale timers/seeks or duplicate listeners.

11.1 Independent error-detection control

Add one test-build-only defective interpreter that deliberately omits the consumption of Free on Put (or equivalently injects a faulty incidence delta). The negative-control test must demonstrate a violated capacity/accounting invariant on a small targeted trace. The shipped code contains no runtime defect switch. Also assert that enabling all normal checks in the test build produces exactly the production interpreter’s results.

11.2 Geometry and performance targets

At the default 100% scene scale, all 20 places, 18 transition bars and 50 arcs must be distinguishable at the selected focus; important labels must not overlap; the two resource hubs must have identifiable incoming/outgoing arcs. Phase 0 pins tolerances for label bounding boxes and line overlap from actual rendered nodes. Native BFS and all three scripted traces should finish comfortably below test-suite time budgets; CDP should exercise one event at a time, not rely on wall-clock sleeps.

12. Implementation plan and acceptance gates

PHASE	SCOPE	EXIT GATE
0 — Probe	Ring prototype: 20 places, 18 gates, 50 arcs. Measure corrected Take anchors, Put return loops, both hubs and extreme spokes, shared-side pinning and overlays.	Real-browser evidence: Free=0.25 / Used=0.75 at both gates, no ClockOnRectangle, no spoke grazes opposite hub, measured minimum clearances and view stability.
1 — Petri core	Typed net, enabling/firing, conservation, reverse BFS and 50/28/30-firing fixtures.	Exactly 5,832 reachable/reverse-reachable markings, zero dead markings, §5 snapshots/indices pinned; no DOM dependency.
2 — Presenter	Build one Scene, token/status overlays and inspector; change only trace/projection on scenario switch.	Full/diff parity, same node/edge elements and IDs, identical zoom/pan and valid focus after switching; no extra SVG nodes/edges.
3 — Interaction	Three-script selection; Manual/Return, Fire selected, mouse/keyboard/touch firing, seek/checkpoints, nonlooping playback and discard notice.	Fork restored, discard announced, manual live notice per fire, Play boundaries exact, one firing per activation and stale actions ignored.
4 — Resilience	Dirty repair including failed in-place trace switch, teardown, deterministic clock and CDP suite; docs.	Failure injection, intact view/focus/DOM identity, lifecycle, forced-colours and keyboard checks pass.

12.1 Definition of done

A reviewer can use Full (24), Blocked (26) and Released (27) in Walkthrough; see the last-slot conflict at Contention 25–27; visit Interleaving 8 and 14 to see simultaneous Busy/Ready/Processing work; and enter Manual from any checkpoint, fire an enabled transition, branch backwards, then return to the exact originating script index. Capacity, actor and event counts stay correct in all directions.

12.2 Non-goals and likely follow-ons

Do not add FIFO queue ordering, item identities/coloured tokens, timed/stochastic firings, actor-count editor, inhibitor arcs, deadlock synthesizer, maximum-throughput simulation or a general-purpose Petri-net modelling API in this release. Follow-ons may introduce timed scheduling or coloured tokens only once the untimed capacity invariants and renderer behaviour are proven.

> **Design approval gate.** Phase 0 must capture measured gate bounds and both hubs, a view of ring return loops, evidence that Take bar endpoints keep Free above Used, and the minimum clearance from the extreme hub spokes to the opposite hub. Also verify overlay access on a **single retained Scene** and that scenario projection updates do not disturb its view or focus. If the overview is too dense, revise geometry/emphasis, not incidence or fixtures.

Appendix A. Exact scenario-generation recipes

The fixed traces are defined in transition names; every push below means fire the named transition against the current marking, check invariants and append one event plus one snapshot. No silent “fill buffer” mutation is allowed.

A.1 Walkthrough: 50 firings

```
for k in 0..8 { i = k % 4;
    push(Start[i]); push(Finish[i]); push(Put[i]); }  // 24
push(Start[0]); push(Finish[0]);                      // 26
push(Take[0]); push(Put[0]); push(Complete[0]);    // 29
for k in 0..8 { j = k % 3;
    push(Take[j]); push(Complete[j]); }            // 45
push(Start[1]); push(Finish[1]); push(Put[1]);    // 48
push(Take[1]); push(Complete[1]);                 // 50
```

A.2 Contention: 28 firings

```
for k in 0..7 { i = k % 4;
    push(Start[i]); push(Finish[i]); push(Put[i]); }  // 21
push(Start[0]); push(Finish[0]);                        // 23
push(Start[1]); push(Finish[1]);                        // 25
push(Put[0]); push(Take[0]); push(Put[1]);            // 28
```

A.3 Interleaving: 30 firings

```
push(Start[0]); push(Start[1]); push(Start[2]);
push(Finish[0]); push(Finish[1]); push(Put[0]);
push(Take[0]); push(Start[3]); push(Finish[2]);
push(Put[1]); push(Take[1]); push(Finish[3]);
push(Put[2]); push(Take[2]); push(Put[3]);
push(Complete[1]); push(Start[0]); push(Start[1]);
push(Finish[0]); push(Finish[1]); push(Put[1]);
push(Take[1]); push(Put[0]); push(Complete[0]);
push(Complete[2]); push(Take[0]); push(Complete[1]);
push(Take[2]); push(Complete[0]); push(Complete[2]);
```

Assert snapshot 30 equals snapshot 0 **as a marking**; historical cumulative counters differ, as they should. Assert at snapshot 8 producers IRBB/consumers PII, at 11 IIRB/PPI, at 14 IIIR/PPP, and at 22 RIII/PPP.

Named checkpoints are explicit snapshot-array indices from §5.4, not event indices. In Manual they are disabled; no implicit mapping from script milestones into manual branch indices is permitted. There are no synthetic waiting events. A blocked status is a property of a marking.

Appendix B. Repository and formal references

Repository baseline reviewed on 4 November 2026: `ChrisWhealy/svg-dom-graph`, Cargo package version **0.2.47** on the default branch. The implementation should recheck those file/API details at Phase 0 as the repository evolves.

1. demo-app/src/lib.rs — gallery registration, demo module conventions, standalone windows and source-file embedding.

2. docs/library/ellipse-nodes.md — circle nodes, clock connectors and anchor limitations.

3. docs/library/connectors.md — arrows, side/position anchors and routing constraints.

4. docs/demo/edmonds-karp.md — pure trace/projection split, focus, dirty repair, real-browser tests.

5. docs/demo/n-queens.md — deterministic clocks, seek and playback lifecycle.

6. Petri Net Theory and the Modeling of Systems, Chapter 2 — ordinary P/T enabling and firing rule.

Appendix C. Revision 2 decision record

REVIEW FINDING	NORMATIVE RESOLUTION
Missing interleaving	Third 30-event script, 31 snapshots, five named checkpoints, exact state fixtures (§5, A.3).
BFS bound / reversibility	Require exactly 5,832 reachable states; all 5,832 can return to M0; no dead states (§4, §11).
Four arcs at Put/Take bar	Forced sides plus 0.25/0.75 fractional side positions on rectangles, measured Phase 0 (§6).
Layout crossings and return	Local actor rings, distinct buffer corridors; inspected hub/return geometry and focused fallback (§6).
Manual-mode ambiguity	Explicit Enter Manual, Return to script, Fire selected, branch indexing, checkpoints unavailable and Play disabled (§7).
Checkpoint indices	Exact mappings in §5.4 for all three scripts.
Firing ergonomics	Single click inspects; manual double-click or Enter/Space fires once; Fire selected alternative (§7).
Figures / playback	Consistent producer / Free over Used / consumer grouping; no looping at script end; default 2 events/s.

Appendix D. Revision 3 decisions

REVIEW ISSUE	RESOLUTION / ACCEPTANCE TEST
Take anchors cross	Take Used input moved to West@0.75 and Free output to West@0.25; same upper-Free/lower-Used convention as Put (§6.3).
Unnecessary scene rebuild	Keep one Scene and 50 arcs; scenario switch changes only trace/projection; stable DOM IDs, zoom/pan and focus (§9.3).
Keyboard / touch safety	Selection inert; keyboard Enter/Space deliberately fires in Manual. Touch taps to select and Fire selected. Announce every successful manual fire (§7).
Silent branch discard	Selecting a scripted scenario from Manual emits a visible/polite “Manual branch discarded” notice (§7.1).
Interleaving coverage	Used never exceeds 1; saturation/empty-buffer proof is furnished by the other fixtures (§5.3).
Hub spokes at extremes	Phase 0 records opposite-hub clearances for all spokes; intersections/grazing fail before Phase 1 (§6.4).

End of specification · Revision 3